

Manga Translator

Kiến trúc hệ thống & cây thư mục — Python · FastAPI · ONNX

Phân tích repo mã nguồn mở (2026)

Nội dung

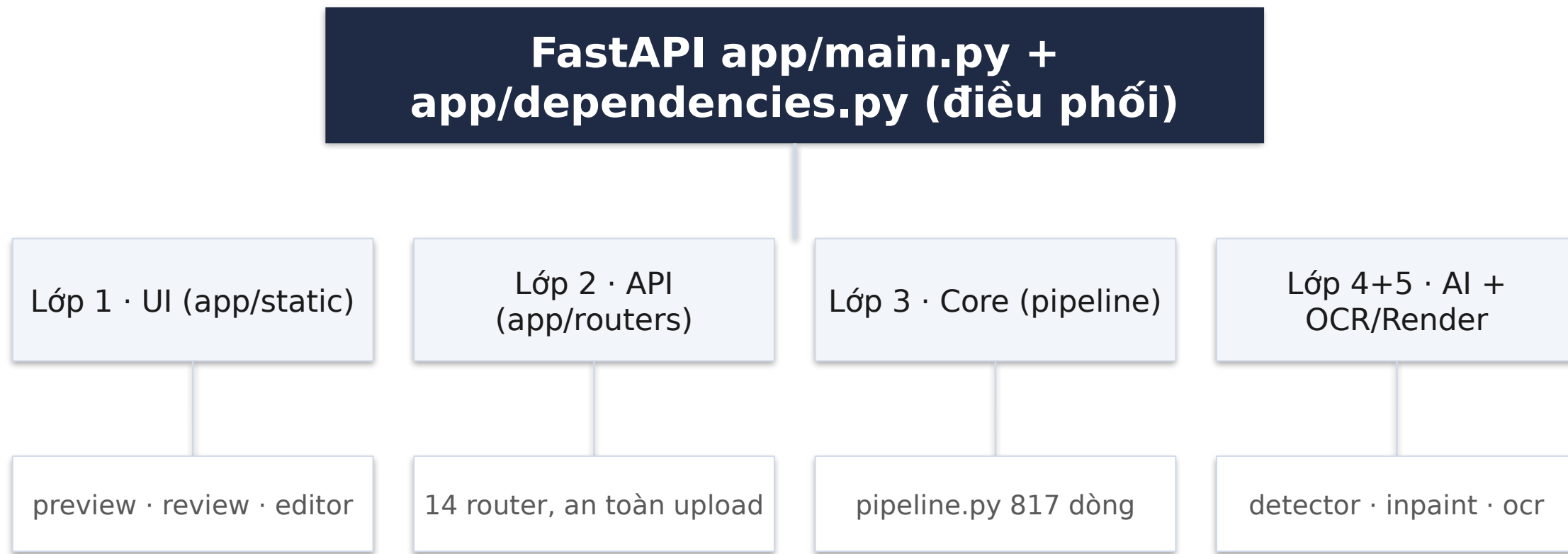
- 01** Thư mục và vai trò
- 02** Sơ đồ kiến trúc tổng quát
- 03** Đi sâu 5 lớp
- 04** Luồng xử lý một chương
- 05** Điểm mạnh, rủi ro, kết luận

Thư mục top-level: vai trò

Thư mục	Vai trò
app/	Toàn bộ ứng dụng: FastAPI, pipeline, detector, OCR, render, UI tĩnh
models/ · data/	ONNX Kiuyha; data: raw, processed, output, temp
docs/ · tests/ · scripts/	ARCHITECTURE.md; pytest; run.py, dev.py, dev_backend.py

Nguồn: kết quả list_dir repo gốc (2026-07)

Kiến trúc tổng quát 5 lớp



Mối ghép: HTTP/JSON ở giữa UI và Core

Lớp 1 — Frontend: app/static

UI 3 màn hình

- preview.js: xem trang đã slice
- review.js: chỉnh box + brush
- editor.js: dịch + font

Công nghệ & bootstrap

- Vanilla JS, không framework
- api.js: bọc mọi /api/*
- main.js: job poll 700ms

Lớp 2 — API: app/routers

14 router (14 file)

- chapters.py: tạo chương + job
- ocr.py · translation.py: OCR/dịch
- export.py: preflight + ZIP

Hàng rào an toàn

- security.py: validate_chapter_id
- MAX_UPLOAD: ≤500MB
- render_commit: snapshot atomic

Lớp 3 — Core: pipeline + dependencies

Orchestrator (pipeline.py)

- download_chapter(): tải + slice
- process_pages(): luồng xthread
- pipeline_editing: sửa box, mask

Sân chơi an toàn

- dependencies.py: singleton
- manifest_utils: file lock + atomic
- security.py: chặn SSRF/traversal

Lớp 4 — AI: detector + inpaint

Phát hiện & xóa chữ

- KiuyhaTextDetector (ONNX, 1280px)
- boxes.py: BubbleBox + NMS cuối
- mask_builder: mask học + fallback

LaMa inpaint (CPU)

- lama_inpainter: crop + tiled 512^2
- crop đơn sắc: tô màu median
- chạy song song với OCR

Lớp 5 — OCR · Downloader · Render

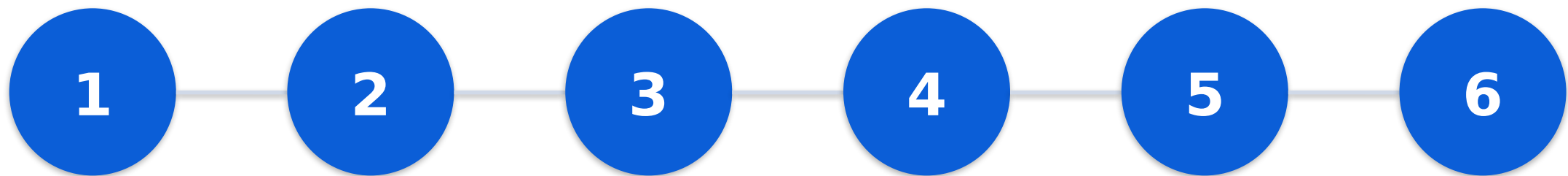
OCR + tải + chèn chữ

- MultiLangOCR: ja/en/zh/ko
- registry: static → Playwright dự phòng
- text_renderer: font 6-48px

Và tùy chọn

- PaddleV6OCR: tier small/medium
- slicer: cắt webtoon dài
- OCR commit dưới manifest lock

Luồng: từ tải về đến xuất file



POST /api/chapter ·registry: static → validate URL (SSRF) Playwright slicer → raw/{id}/sliced/ detector + LaMa → clean_*.png OCR + render → data/output preflight → chapter_{id}.zip

Mỗi bước: đầu vào / đầu ra

Bước	Đầu vào → Đầu ra
1. Validate	URL (hoặc files) → manifest + download_job
2. Slicing	raw/*.png → sliced/*.png
3. Detector + LaMa	sliced/*.png → clean_*.png
4. OCR + render	clean_*.png → OCR + output
5. Export	manifest + output → ZIP

Nguồn: app/pipeline.py, app/routers

Điểm mạnh chính

3 lớp

Rõ & độc lập

UI / API / Core tách nhau, API
hợp nhất một cổng

**14
router**

Adapter tải

Static, JS, upload ảnh/ZIP/CBZ

**100%
CPU**

ONNX + Paddle

Kiuyha, LaMa, CTD, OCR —
không cần GPU

Rủi ro chính

Crawler

Thay đổi HTML

Site đổi HTML là adapter hỏng;
phụ thuộc Playwright

pipeline .py

817 dòng riêng

Chưa tính phụ thuộc, lock phức
tạp, E2E mỏng

Nhiều file tạm

raw → output

raw, processed, output, temp để
lịch manifest

Kết luận

**Manga Translator = FastAPI +
pipeline điều phối + ONNX CPU
+ manifest lock**

Nguồn: docs/ARCHITECTURE.md (6/6/2026)

Cảm ơn

Manga Translator — kiến trúc hệ thống & phân tích repo